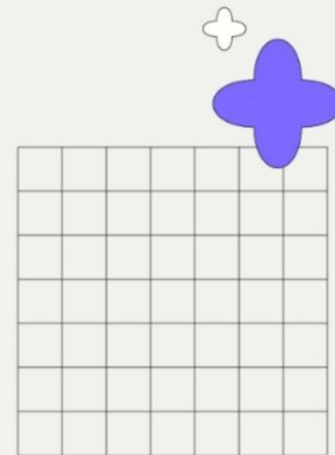
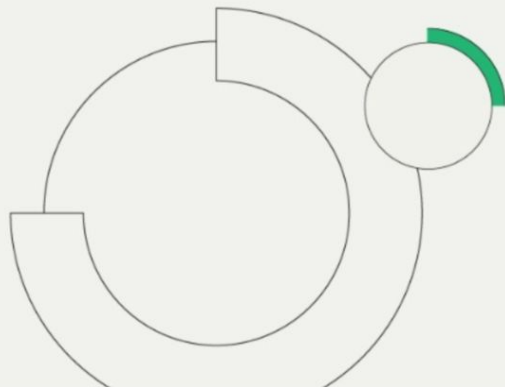
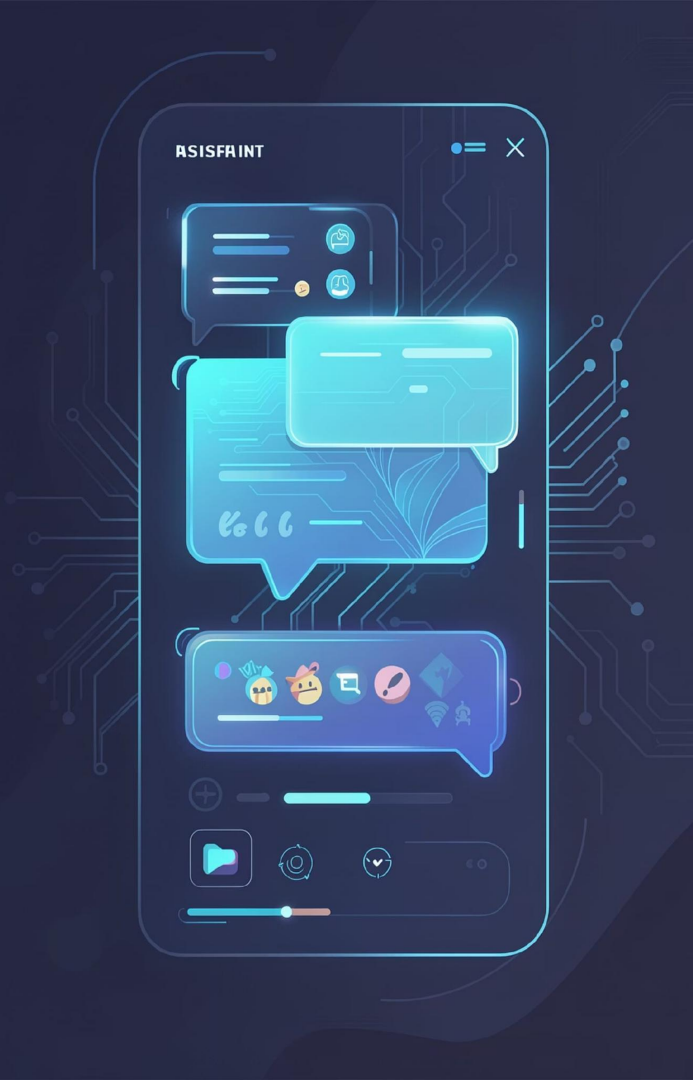




שיעור 11 - צ'טבוט חכם ארכיטקטורת מערכת

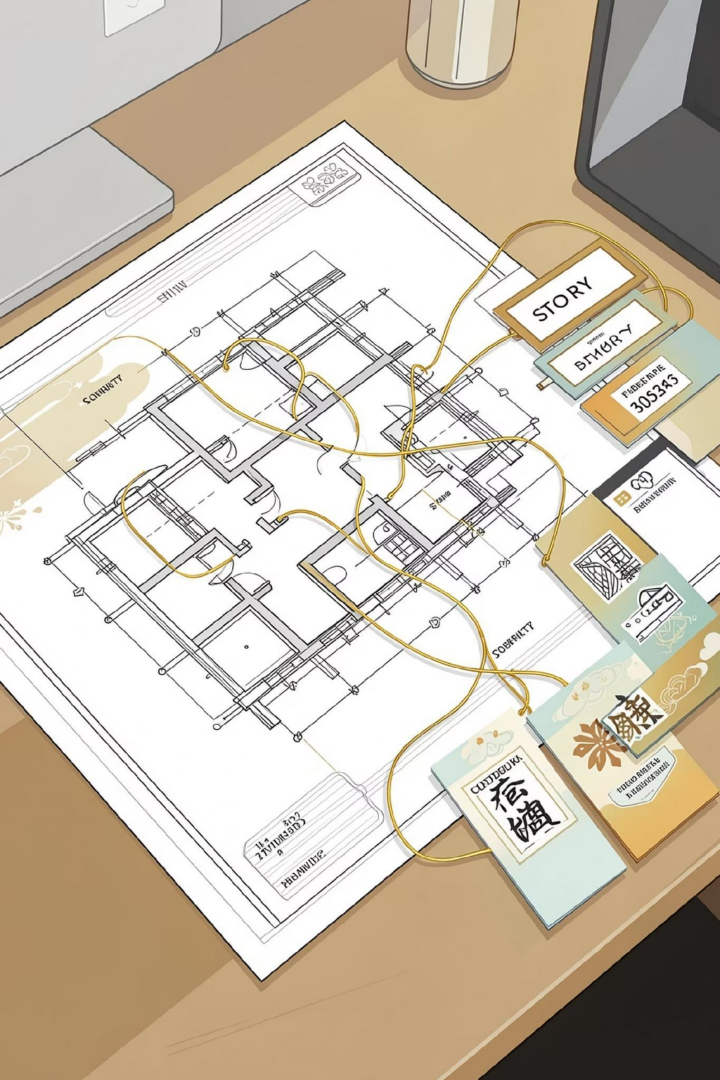


שיעור 11:

הכנסת העוזר החכם למערכת

המערכת מקבלת AI שעונה לעובדים, ואנחנו מתחילים לתכנן מה שמאחורי הקלעים

מערכת דיווח תקלות ללא פחד | קורס מיישם AI



מה נעשה היום

עד עכשיו כתבנו מה המערכת צריכה לעשות.
התחלנו ליצור MVP כהתחלה לבניית המערכת.

היום נכניס ל MVP עוזר AI שיענה על השאלות במסמכים שיצרנו לדוגמה "מי יראה שדיווחתי?"

חוט השני: התרחישים שכתבתם בשלב הסיפורים הם שאלות הבדיקה של המערכת. היום נראה איך בודקת את עצמה

חוק יסוד - קודם מסמך, רק אז בנייה

נדרשנו להוסיף למערכת צ'אטבוט חכם
שיענה על שאלות הקשורות למערכת.
הפיתוי: לבנות אותו מיד.

העיקרון שקבענו:

הדיווח על כל תקלה נסגר אחרי אימות
בשטח ועדכון המדווח.

החלנו את אותו עיקרון בבניית המערכת:
יכולת נכנסת רק אחרי שיש מסמך
החלטה מאושר ועדכון ה-PRD.



פוגשים את העוזר

הבסיס של הכל

דפי נהלים שהעלינו לעוזר-
זה כל מה שהעוזר יודע.



שאלה שנדחית

"מה שעות הקפיטריה?"

← העוזר: מצטער, אבל אי לי
תשובה, זה לא חלק
מהנהלים שלי.

שאלה

"מי רואה שדיווחתי?"

← תשובה חייבת להיות
אמיתית ומרגיעה, מתוך נוהל
האנונימיות.

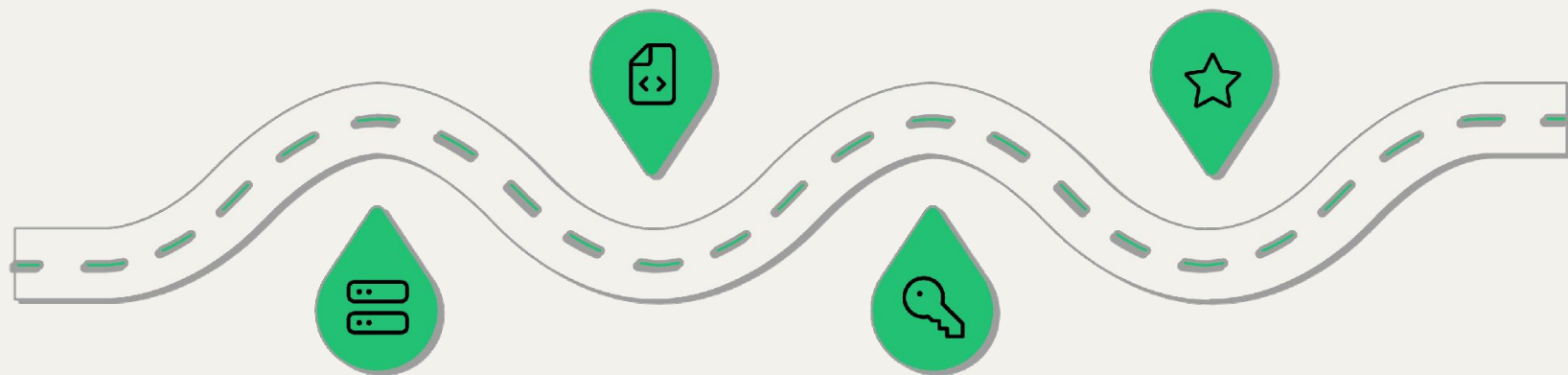
הצ'אטבוט הוא פרומפט טוב שקיבל ממשק משתמש. החלק הקשה, לכתוב פרומפט, על זה נתאמן היום.

איך המסך שלנו מדבר עם ה-AI

כשעובד מקליד "מי רואה את מה שדיווחתי?" ולוחץ שליחה, מה בדיוק קורה מאחורי הקלעים בשנייה הזו?

שאלת משתמש:
הבקשה יוצאת
מהמסך שלנו

נוצר טופס/JSON
הנתונים המובנים
והפרמטרים



כתובת API
הנקודה שמקבלת
ומחזירה תשובה

מופעל מפתח API
מזהה סודי המאמת
את הבקשה

כל בקשה ל-AI מורכבת משלושה חלקים: מפתח, טופס, וכתובת. ועוברת ארבע תחנות במערכת שלנו.
בנוסף החלטה: באיזו דרגת מודל נשתמש (חינמי, זול, יקר)



תחנה 1: המפתח AI-KEY

מודל ה-AI הוא שירות בתשלום של ספק חיצוני.
איך הספק יודע שהבקשה מאיתנו ואת מי לחייב?

API KEY - מפתח: מחרוזת סודית שמצורפת לכל בקשה.

ביוצור - אנחנו שומרים את זהות המדווח בשכבה מוצפנת נפרדת, לא בגוף הדיווח שכולם רואים.

מפתח ה-API מקבל את אותו יחס: הוא סודי, ולעולם לא נכתב בקוד שכל פותח קובץ רואה. לכתיבת סוד בקוד גלוי קוראים **Hard-Coded**, ובמונחי המערכת שלנו: חשיפת זהות על גוף הדיווח.

כרגע נשתמש בקוד חימני בלבד

בקובץ שלנו: המפתח מוקלד בכל פעם מחדש, חי בזיכרון, ונעלם כשסוגרים.

תחנה 2: הטופס, ה-JSON

סיפור משתמש 2: העובד שולח צילום והודעה, וה-AI ממלא את פרטי הדיווח: מיקום, סוג תקלה, דחיפות, מצב דיווח. מעבירים שדות קבועים: הדשבורד צריך לספור "**כמה תקלות בדחיפות גבוהה בחודש**". הספירה אפשרית רק כשלדחיפות יש שדה קבוע.

דיווח תקלה JSON

```
}  
"מיקום": "קו ייצור 3",  
"סוג_תקלה": "נזילה",  
"דחיפות": "גבוהה",  
"מצב": "חדש"  
{
```

המבנה הזה, שדה: ערך בתוך סוגריים מסולסלים, נקרא **JSON**.

שאלה ותשובה לשאלה

גם השאלה לצ'ט נשלחת באותו פורמט

תשובה לצ'ט - JSON

```
}  
  "מודל": "gemini-flash",  
  "תשובה": "אף אחד לא רואה שדיווחת.  
הזהות שלך גלויה רק למנהל המערכת ולך בלבד.  
מנהלים וצוות התחזוקה רואים את התקלה בלבד,  
ללא שם.",  
  "טוקנים_בשאלה": 210,  
  "טוקנים_בתשובה": 38  
{
```

שאלה לצ'ט - JSON

```
}  
  "מודל": "gemini-flash",  
  "מקסימום_תווים": 300,  
  "שאלה": "מי רואה שדיווחתי?"  
{
```

מי קובע את הפורמט?

טופס הדיווח שלנו

אנחנו קבענו, כשאפיינו את
המערכת.

טופס של הספק

הספק קובע, ומפרסם בדף
שנקרא **תיעוד**.

התפקיד שלנו

אנחנו פועלים לפי הנוהל שלו,
כמו שהעובדים ממלאים דיווח
לפי הנוהל הנדרש.

בפועל: מבקשים מה-AI למלא את הפורמט לפי התיעוד או המבנה המועבר מהספק.
התפקיד שלנו: להבין מה כתוב כדי לבדוק שיצר את המידע נכון.

תחנה 3: הכתובת הרשמית



במערכת שלנו: כל דיווח נכנס דרך ערוץ רשמי אחד ("**דיווח חדש**"), כדי שכל דיווח יירשם ויטופל. אחרת דיווחים הולכים לאיבוד.

לספק ה-AI אותו כלל: כל הבקשות נכנסות דרך כתובת רשמית אחת. הכתובת הזו נקראת **API**.

מה חוזר? טופס תשובה, ב JSON:

התשובה, ושדה שמראה כמה נצרך.

סיכום: מפתח + טופס (JSON) + כתובת (API) = כל שיחה עם AI.

תחנה 4: איזו דרגת מודל? כמה זה עולה לנו

אצל ספקי ה-AI יש מודלים בדרגות תמחור שונות: זולים ומהירים, ויקרים ומעמיקים. ברירת המחדל החכמה: מודל זול (למשל Gemini בגרסה החינמית) עונה מצוין על רוב השאלות.

מודל זול: שגרה

שאלות רגילות, מענה מהיר,
עלות נמוכה. כמו טיפול שוטף
בדיווח רגיל.

מודל יקר: לניתוח מורכב

ניתוח מגמות, שיקול דעת
עמוק. כמו משאבים קריטיים
לתקלה דחופה.

נתב: ההחלטה האוטומטית

מנגנון שמחליט אוטומטית
לאיזה מודל לשלוח כל שאלה.

השאלה האמיתית שעלינו לשאול: מתי צריך את המודל היקר?

החשוב: יחידת החיוב היא חלק ממילה (בערך שלושת רבעי מילה), ושמה **טוקן**.

בקבוצות, על הפרויקט שלכם. צרו את מסמכי המידע והוסיפו יכולת העלאת קבצים:

01

דף החלטה לעוזר

שלושה משפטים:
מה עונה, ממה, מה
נדחה.

02

טופס JSON על
נייר

שאלה אמיתית של
משתמש מהמערכת
שלכם.

03

שלוש שאלות
לדוגמה

מהמערכת שלכם,
ולכל אחת: מודל זול
או יקר, ולמה.

04

שמרו בפרויקט ועדכנו
שמרו את דף ההחלטה בפרויקט והריצו עדכון PRD.



עכשיו תורכם

הגדירו את מסמכי המידע לפרויקט שלכם, ואת מסך העלאת הקבצים



תרגיל שני: הוספת המפתח ומסך ההגדרות



הפיקו מפתח משלכם

לחצו "Create"
"API key"
המפתח מתחיל
ב-Alza ומופיע
פעם אחת.

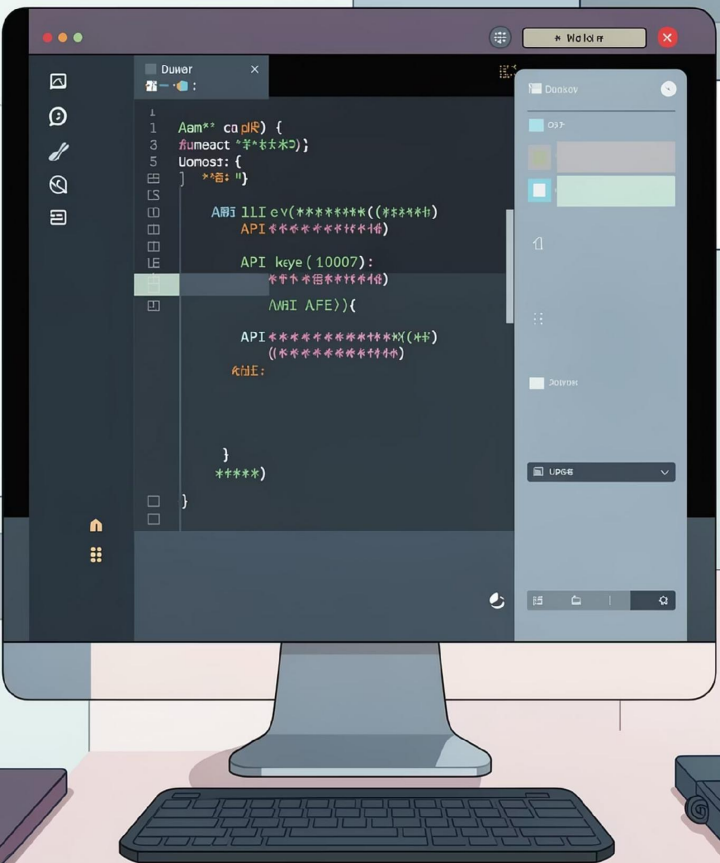
התחברו עם
חשבון גוגל שלכם

היכנסו
Google AI-7
Studio

aistudio.google.com/apikey

העתיקו את הקוד
אל תשמרו אותו במסמכי הפרויקט, וגם לא בקוד.

זו השכבה החינמית של Gemini:
ללא כרטיס אשראי, עם מכסה יומית מתחדשת.



תרגיל 2: בנו את מסך קליטת המפתח

המשימה: להוסיף ל-MLP מסך שקולט את המפתח.

לאן הוא נכנס? בטח לא לטאב תוכן.

עובד שנכנס לדווח לא צריך לראות שדה מפתח.

המקום הנכון: הגדרות מערכת. גלגל שיניים בפינה, ובתוכו נמצא את כול הקונפיגורציה.

זוהי החלטת ארכיטקטורה ראשונה: הפרדה בין הגדרות לתוכן.



מה נכניס להגדרות בשלב זה

1

חיבור AI

רשימת הספקים, קישור להנפקת מפתח לכל אחד, שדה מוסתר, וכפתור בדיקה.

3

מקורות מידע

העלאת הנהלים והנתונים.

2

היררכיית שימוש

באיזה ספק ואיזה מודל שלו משתמשים לדרגה זולה ואיזה לדרגה יקרה. (סעיף 14).

4

מצב המערכת

מי מחובר, מה טעון, וכמה טוקנים נצרכו.

תרגיל 3: הזינו ובדקו

1

פתחו את ההגדרות
הדביקו את המפתח
שהנפקתם, ולחצו בדיקה
וחיבור.

2

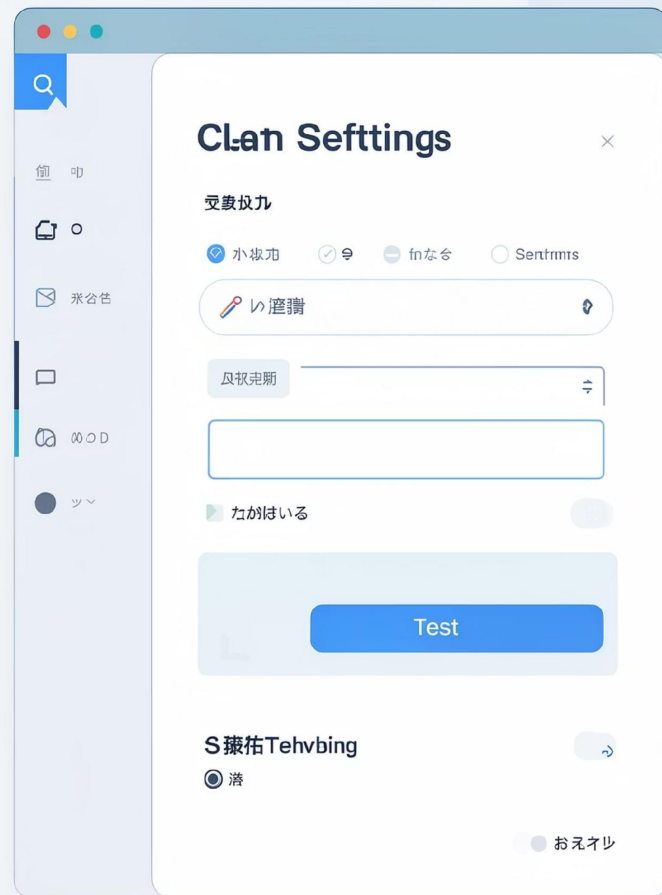
המערכת שולחת בקשת
ניסיון אמיתית לספק
מפתח תקין מתחבר, ומפתח
שגוי מציג הודעת שגיאה
מהספק.

3

טענו את ארבעת הנהלים
רק אז מסך העוזר נפתח.

4

בדקו: רעננו את הדף
המפתח נעלם. זה לא באג, זו
ההגנה. בכל התחברות
העתיקו אותו מחדש



"להצפין את המפתח"



הגנה נראית לעין

השדה מוסתר בנקודות. זוהי הגנה מפני מי שמסתכל על המסך, וזה חשוב במיוחד בהדגמה מול כיתה.

אבל הצפנה ושמירה בדפדפן היא אשליה: כדי לפענח, מפתח ההצפנה חייב לחיות באותו דפדפן, בקוד שכולם יכולים לראות.



ההגנה האמיתית לפרוטוקול שלנו

לא לשמור. המפתח חי רק בזיכרון ונמחק ברענון/סגירה.

ההגנה החזקה באמת מגיעה בייצור:

המפתח עובר לצד שרת, והמשתמש לעולם לא יכול לראות אותו.

עכשיו תורכם

הוסיפו את המפתח ואת מסך ההגדרות. העבירו את העלאת הקבצים למסך ההגדרות



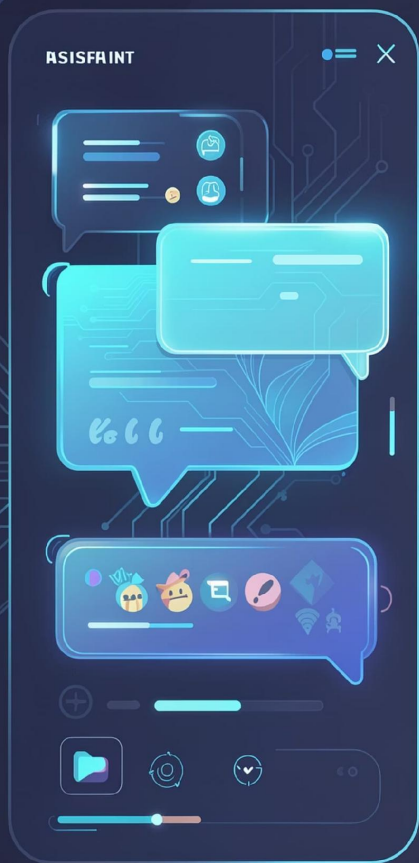
חלק ב: התהליך

נטפל ב"ירידה לשטח" דיגיטלית על סיפור משתמש מספר 2

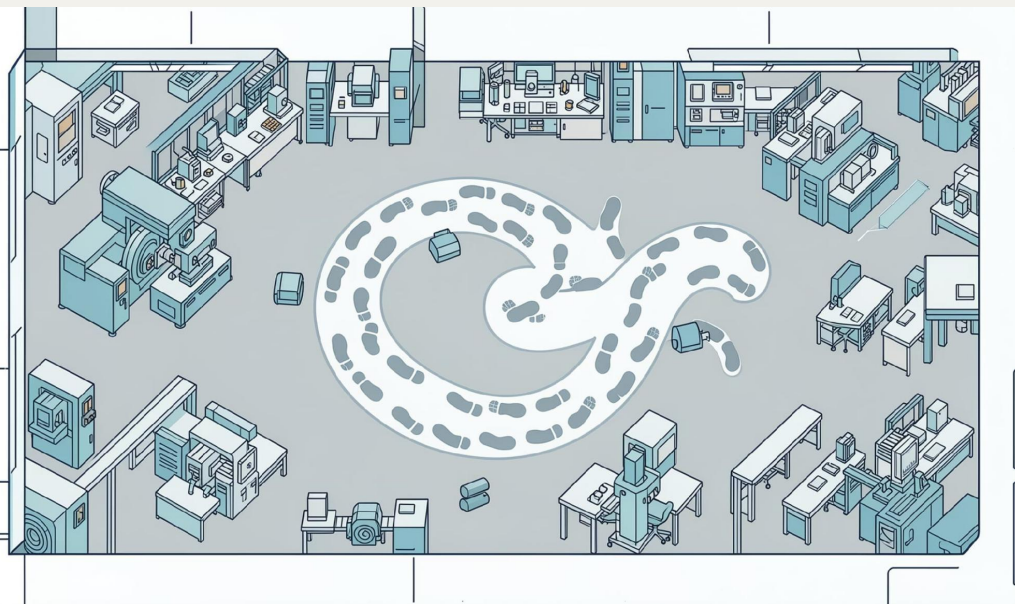
1. העובד שולח צילום והודעה קולית קצרה לבוט הדיווח בוואטסאפ או בטלגרם,
2. ה־AI
 - a. מנסח את תיאור האירוע
 - b. מסווג את סוג התקלה
 - c. מציע דירוג חומרה
 - d. מנתב את הדיווח ליעדו
 - e. מחזיר את הניסוח לאותה שיחה לאישור או לתיקון בהודעה קולית נוספת.

מספר הטלפון של השולח מופרד מתוכן הדיווח מיד בקליטה ונשמר בשכבה המוצפנת

הבוט שואל בפתיחה "דיווח אנונימי?" כדי להפעיל את כלל האנונימיות המלא.



חלק ב: התהליך



מה שנבנה זהו המסלול הטכנולוגי של הדרישה, וזו ליבת עבודת הארכיטקט.

מושג מעולם ה-LEAN שדברנו עליו בפרויקט: **ירידה לשטח**.

עלינו לבדוק את התהליך ולראות שהוא עובד.

נבדוק את התהליך מרגע איתור התקלה ועד לסיום הרישום.

ארבע עמודות: מה קורה, מי מבצע, מה עובר, מה אם נכשל?

מה עם רכיב או כשל שה-AI מציע ושלא כתוב בסיפור?

מסומן [חדש] או [טרם נקבע].

מה שאין לו מקור במסמכים לא נכנס בלי הכרעת אדם.

חלק ב: התהליך

תחילה נבדוק האם האפליקציה שה-AI יצר אכן עונה
כל כל הדרישות

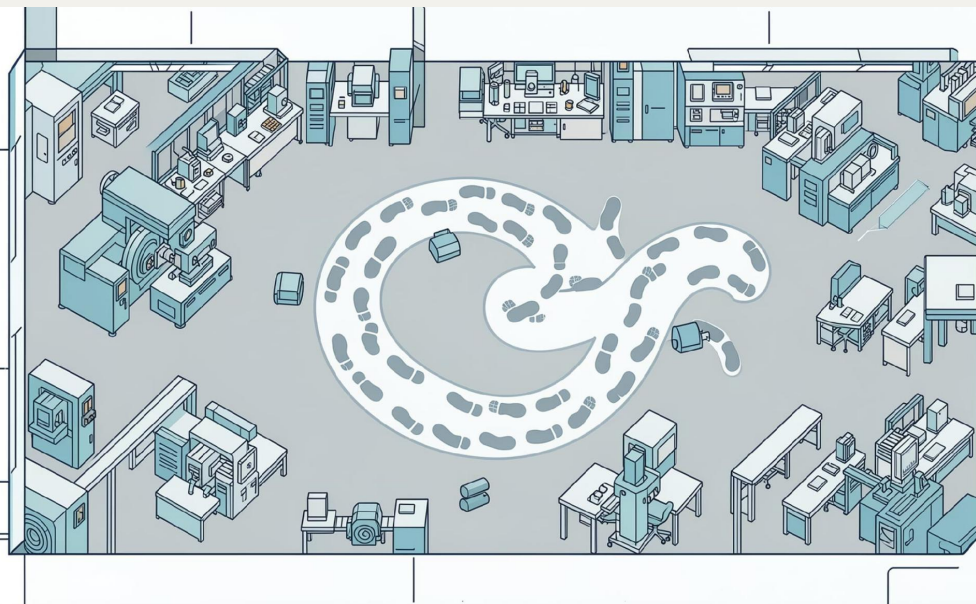
לדוגמה - מסך הקליטה במערכת ניהול תקלות
מתעלם מהדרישה לשלוח באמצעות טלגרם או
וואצאפ.

המערכת בנויה להקלדה ידנית של התקלה

האם נאפשר גם הקלדה?

האם רק טלגרם או וואצאפ?

יש לקבל החלטה ולתקן.



עמודת הכשלים - לב המערכת

עמודת "ואם הדיווח נכשל" נגזרת מהחריגים שכתבנו בתרחישים.
כל חריג הופך כאן לדרישת שחייבים להכניס לבנייה.

עמודת
הכשלים
היא לב
המערכת

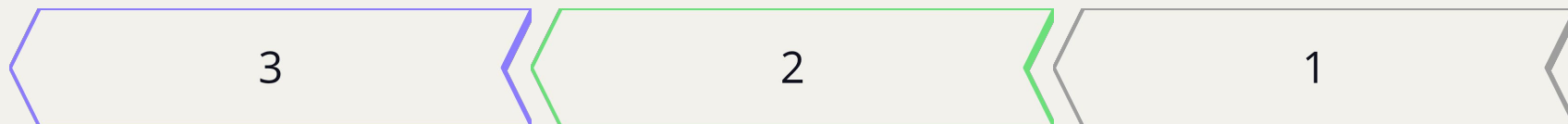
עובד שהדיווח
שלו נעלם
פעם אחת
לא ידווח שוב.

מערכת
שנלחמת
בפחד לדווח
לא יכולה
להרשות
לעצמה לאבד
דיווח.



כתבו את הפרומפט שעושה את זה

שינוי כללים: הפעם לא תקבלו פרומפט מוכן. כתבו אותו יחד עם ה-AI בעצמכם.
פרומפט עבודה בנוי בדיוק כמו נוהל עבודה. חמישה חלקים:



הגדרת התוצר
מבנה הפלט.

המשימה
פעילות אחת ברורה.

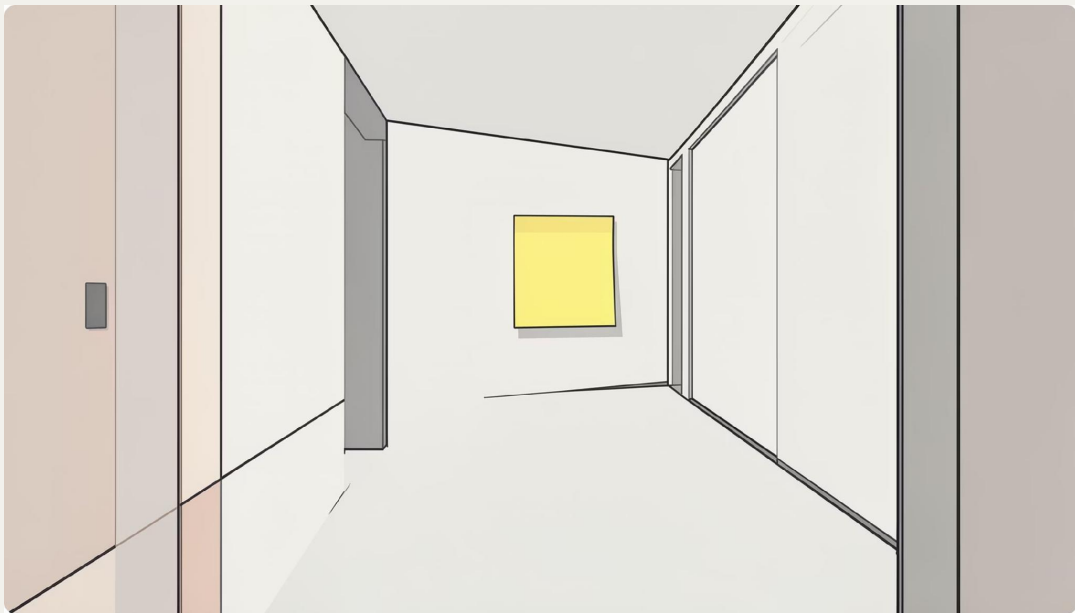
תחולה
ממה ה-AI עובד: המקור.



אישור ממונה
נקודת העצירה.

כללי הבטיחות
חוקי הברזל:
אל תמציא, [טרם נקבע].

מלל חופשי "סתום"



המנעו ממלל חופשי ללא מבנה לדוגמה:
"תסדר את זה".

לא רשום מה, לא לפי מה, אין נקודת
בדיקה, וכל פעם יוצא משהו אחר.

הפרומפטים של המערכת שלנו בנויים כדי
למנוע מלל סתום.

אנחנו עובדים מול ה-AI באותו
סטנדרט שנדרש בארגון:
הכול בנוהל כתוב.

עכשיו תורכם

כתבו את פרומפט לפי הנוהל כולל מה אם נכשל של המערכת שלכם



ההחלטה ההנדסית הראשונה

לכל צעד במסלול, הגדר מה דרגת המימוש? שלוש אפשרויות:

במודל יקר AI

ניתוח מורכב או שיקול דעת
עמוק. למשל ניתוח מגמת
תקלות חוזרות.

במודל זול AI

דורש הבנה פשוטה. למשל
ניסוח תיאור התקלה, סיווג סוג
ודחיפות.

קוד רגיל (ללא AI)

פעולה זהה בכל פעם, ללא
הפעלת שיקול דעת. למשל
שמירת הצילום ורישום הדיווח.

הכלל, עלות כספית: קוד רגיל בלי AI. מודל זול לפני יקר. כל שקל מיותר מכרסם ב-ROI
שהבטחנו להנהלה. הטבלה המסווגת שתבנו מטופלת בסעיף 14 ב-PRD שלכם.



בקבוצות, על סיפור המשתמש המרכזי שלכם. כתבו את הפרומפטים, לפי חמשת חלקי הנוהל.

01

כתבו את פרומפט של המסלול הריצו. בדקו: האם המידע נלקח רק מהמסמכים? רכיבים ללא מקור סומנו?

04

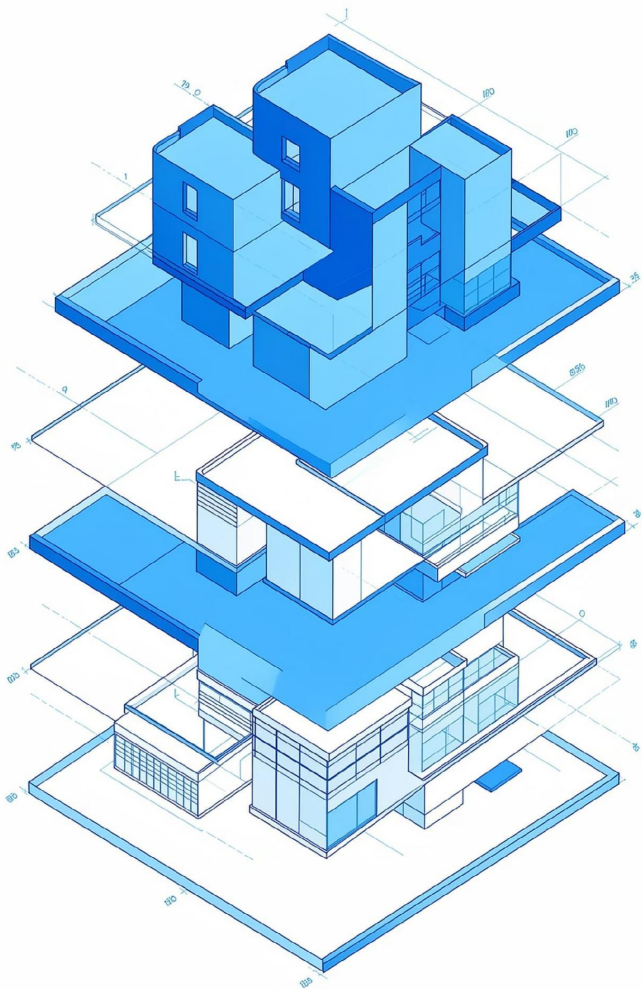
שמרו בפרויקט הטבלה המסווגת, קוד התרשים, והפרומפטים שכתבתם. הפרומפט זה הנכס שלכם.

02

תוצאה שגויה / חלק חסר
מצאו, תקנו, הריצו שוב. בצעו סבבי תיקון נדרשים.

03

כתבו את פרומפט הסיווג
דרגת מימוש לפי סעיף 14, ואת פרומפט התרשים. הריצו ב-draw.io.



ארכיטקטורת מערכת

איך המערכת שלנו בנויה

מערכת דיווח תקלות ללא פחד | קורס מיישם AI

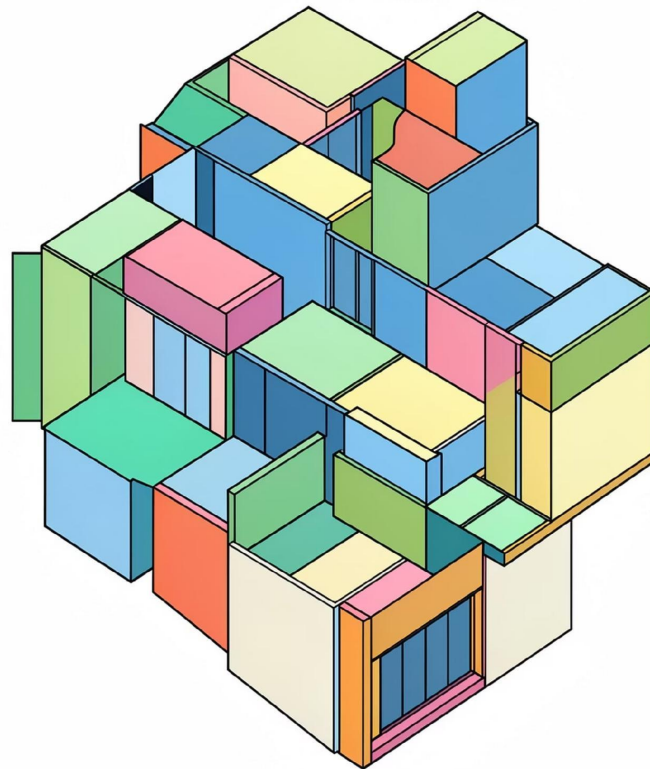
מה זו ארכיטקטורה

בניתם עוזר שעונה לעובדים, שכבת נהלים, ולופ עדכון. שלושה רכיבים אמיתיים.

עכשיו נולדת השאלה מעצמה: איך הם מתחברים למערכת אחת?

ארכיטקטורה היא התשובה: אילו חלקים יש, מי מדבר עם מי, ואיפה עוברים הגבולות.

היום ברמה גבוהה בלבד: קופסאות וחיצים. בשיעורים הבאים נרד לעומק.



המצב היום: הכול בדפדפן



ה-MLP שבניתם רץ כולו בדפדפן.
אין שרת, אין מסד נתונים.

המסכים, המפתחות, הנהלים
ונתוני התקלות:

הכול חי בזיכרון ונמחק ברענון.

הדפדפן מדבר ישירות עם ספקי
ה-AI.

זה בסדר גמור לאב טיפוס, ויש לזה מחיר

שלוש השכבות של מערכת אמיתית

FE: הצד הקדמי

מה שהמשתמש רואה.
המסכים.

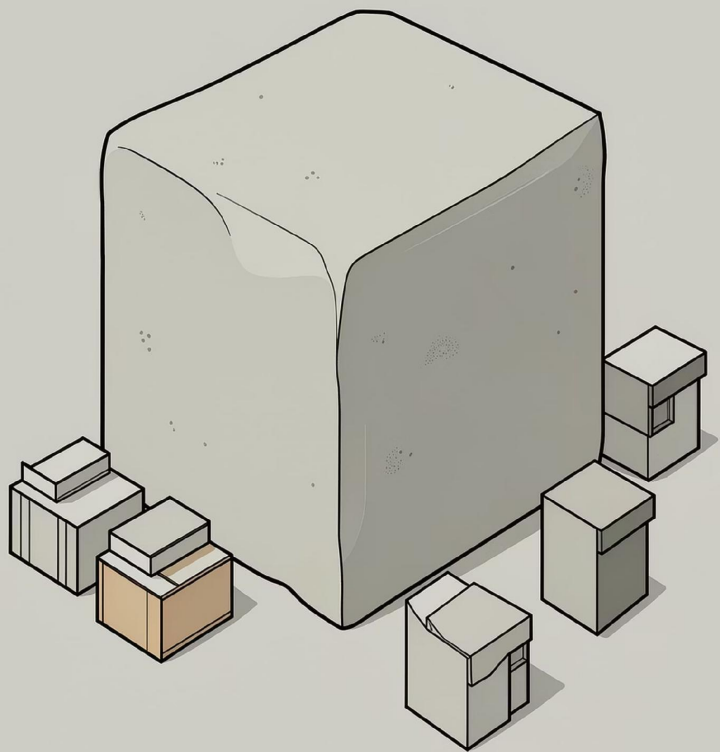
BE: הצד האחורי

שרת המערכת. הלוגיקה,
ההחלטות, והסודות.

DB: מסדי נתונים

איפה המידע נשמר.

התקשורת ביניהם: API + JSON.



למה לא מונוליט?

הפיתוי: לכתוב את כל הקוד במקום אחד. זה עובד, עד שזה מפסיק לעבוד.

כשמערכת נבנית כגוש אחד: כל שינוי קטן מסכן את כל השאר, אי אפשר לפצל עבודה בין אנשים, ואי אפשר להחליף חלק בלי לגעת בשאר.

המערכת שלנו נועדה לצמוח: היום יש ארבעה נהלים ועשרים דיווחים, מחר מפעל שלם

לכן: בונים מודולים.

חלקים נפרדים שמדברים ביניהם בממשקים מוגדרים.

FE: מודולים, ומודול עיצוב אחד

מודול עיצוב:

צבעים, טיפוגרפיה, רכיבי בסיס. מקור אמת אחד לעיצוב, וכל השאר נגזר ממנו.

כל מודול מתחבר למודול העיצוב ולכן כל המערכת נראית בדיוק אותו דבר.

שינוי בצבע? מתבצע במודול עיצוב. כל המסכים מתעדכנים אוטומטית.

מעקב ומונה אישי

דיווח

עוזר חכם

דשבורד

הגדרות

ניתוח

BE: שירותים פונקציונאליים

אורקסטרטור:

- מקבל בקשה
- מזהה את סוגה
- מנתב לשירות הנכון
- מחזיר תשובה.

הוא לא עושה את העבודה, הוא מנהל אותה.

היתרון:

רוצים לשנות את מדיניות העלויות?

נוגעים בשירות AI בלבד.

שאר המערכת לא יודעת שמשו קרה.



שירות זהויות



שירות דיווחים



שירות ידע



שירות AI



שירות התראות

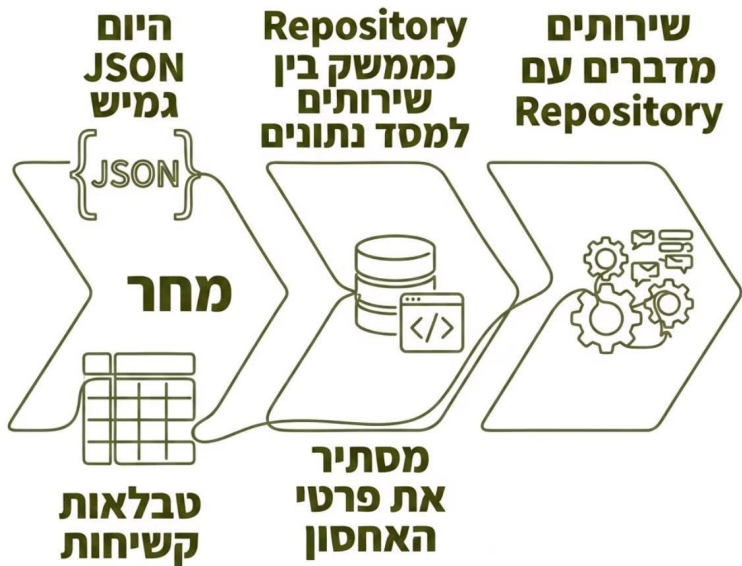
שכבת ה-Repository

השירותים לא ניגשים למסד הנתונים ישירות.
הם מדברים עם שכבת Repository.

- הממשק האחד לגישה לנתונים
- מסתירה מהשירותים איפה ואיך המידע שמור.

למה זה קריטי?

- היום הנתונים סינתטיים אנחנו לא מכירים את המבנה שיגיע ממערכות הארגון
- מסמכי JSON גמישים.
- כשנתחבר למקורות ונכיר את המבנה: נוכל לבנות מבנה טבלאי קשיח.



ההחלפה לא תפגע באף שירות. זו התשתית לצמיחה.

ERD: הישויות והקשרים

הישויות במערכת:

- דיווח
- זהות מוצפנת
- יומן גישה
- מונה איתורים
- לוח מצטיינים
- פעולה מתקנת, משתמש.

ההכרעה המרכזית: על הדיווח יושב טוקן (מזהה) אקראי, ולא שם.

רק שירות הזהויות יודע לקשור בין הטוקן לאדם.

כך ההבטחה "אף אחד לא רואה שדיווחת" עומדת גם אם מישהו מקבל גישה לטבלת הדיווחים.





מה שעדיין לא ידוע?

ערוץ התראות
טרם נקבע.

הרשאות
יגיעו ממערכת ניהול
הרשאות של המפעל, או
שייבנו אצלנו. טרם
נקבע.

בחירת ספק AI
היום פרטנית
בעתיד - תהפוך
ארגונית.

מערכות ארגוניות
לחיבור
טרם נקבע.

ארכיטקטורה מודולרית מייצרת מקום לעתיד, במקום
לנחש אותו.

המחיר של המצב היום

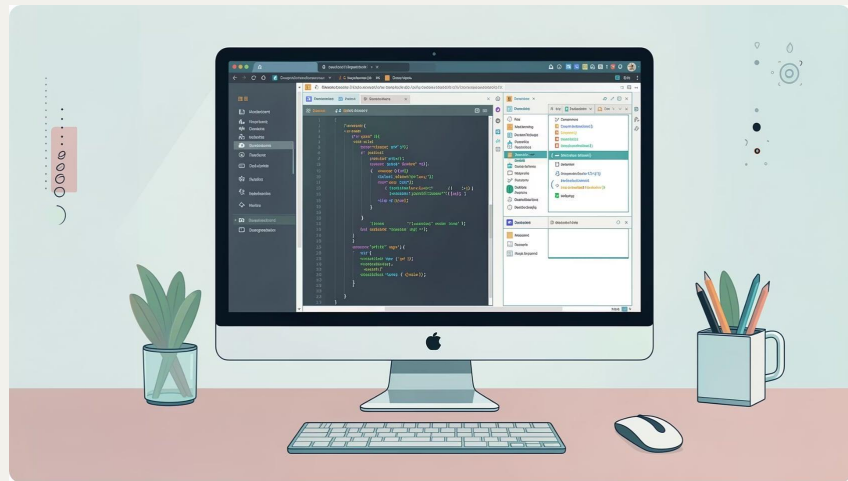
בדפדפן, כל מה שיש בקוד גלוי לכל מי שפותח אותו.
זו עובדה, ולא באג.

לכן המפתח מוזן בזמן ריצה, חי בזיכרון, ונמחק ברענון.

באב טיפוס עם מפתח חינוכי הסיכון מדוד:

מקסימום נבזז את המכסה החינוכית.

במערכת אמיתית, עם חשבון בתשלום וזהויות
מדווחים, זה בלתי אפשרי.



ההפרדה שפותרת את זה

מה שנחשף בהכרח - FE

המסכים והשאלה של המשתמש. לא שמים
בו סוד.

מה שלעולם לא נחשף - BE

מפתחות הספקים, פרומפטי המערכת, מסד
הידע, לוגיקת הניתוב, וזהויות המדווחים.

וזו בדיוק ההחלטה שכבר קיבלתם במערכת: הזהות אינה נכתבת על גוף הדיווח, אלא חיה בשכבה
נפרדת שרק מנהל המערכת ניגש אליה.

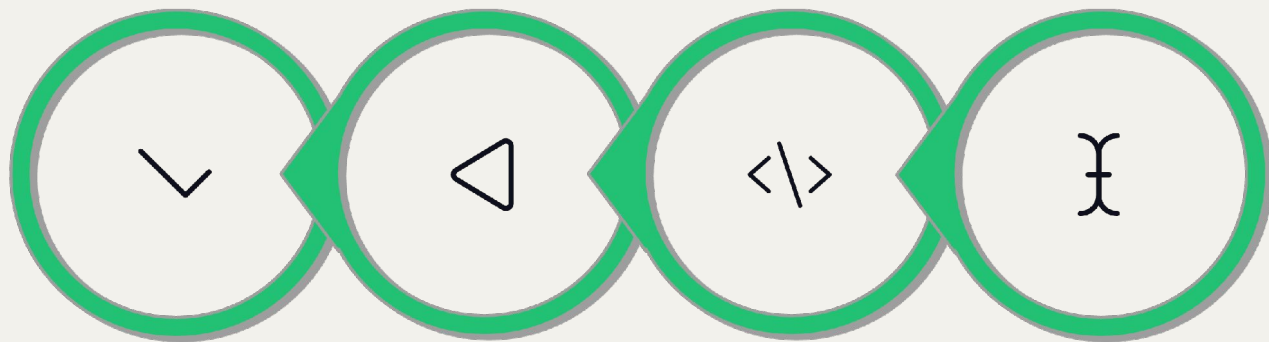
אותו עיקרון, פעם על מדווחים ופעם על סודות המערכת.

MLP מול ייצור: מה משתנה

ייצור ← יעד	MLP ← היום	רכיב
משתנה סביבה בשרת	מוזן במסך ההגדרות	המפתח
FE → שרת שלנו → ספק	מהדפדפן ישירות	הקריאה לספק
בשרת בלבד	בקוד הדפדפן	פרומפט המערכת
מסד נתונים בשרת	ביכרון הדפדפן	מסד הידע
הארגון מחשבון מנוהל	התלמיד מהמכסה החינמית	מי משלם

לצייר במקום להסביר

נסו להסביר טבלה של שבעה צעדים בטלפון לממונה הבטיחות. במקום להסביר, מציירים.



אימות
והתאמה

הרצה
ב-draw.io

יצירת קוד
Mermaid

כתיבת הנחיה

תרשים שעונה על "**מה קורה אחרי מה**" נקרא **תרשים זרימה**: תיבות מסמנות פעולות, מעוינים לנקודות החלטה וחיצים שמחברים ביניהם. זה נוהל ליצירת תרשים זרימה.

כותבים הנחיה שמייצרת קוד Mermaid, מריצים את הקוד ב-draw.io. הכלי - כתיבת ההנחיה.

העבודה שלכם לבדוק: כל תיבה, כל מעוין. ציור שאינו נאמן לטבלה = הנחיה שדורשת תיקון.

תרשימי אב ובנים

תרשים שאינו נכנס למסך אחד, כשל בתפקידו.

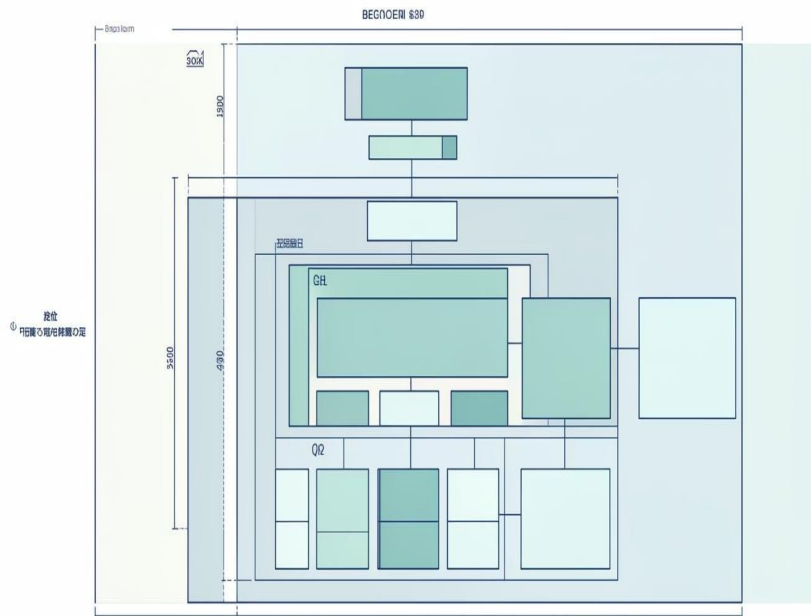
תרשים אב: המסלול הראשי, קופסאות בלבד.

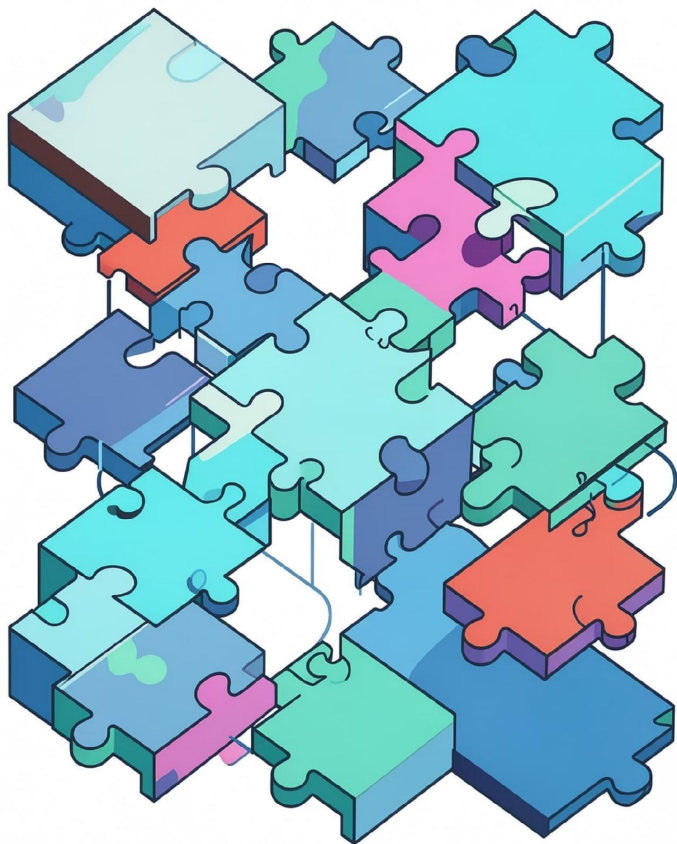
תרשים בן: פירוט של חלק אחד בתרשים האב.

כך בנויים כל התרשימים שלנו:

ארכיטקטורה, ERD, זרימת בקשה, ותהליך העבודה.

והשיטה: כותבים הנחיה שמייצרת קוד Mermaid, ומריצים ב-draw.io. הבדיקה מול המקור היא העבודה שלכם.





מה זה מאפשר לנו

מבנה מודולרי אינו אקדמי. הוא מה שמאפשר לחבר דברים בבלי שהקוד ישבר.

→ שירות התראות מוגדר
אפשר לחבר כלי אוטומציה כמו Make.

→ שירות דיווחים מוגדר
אפשר לחבר מערכת ארגונית.

→ Repository מוגדר
אפשר להחליף אחסון בלי לגעת בלוגיקה.

כל חיבור עתידי נכנס דרך שירות, ולא דרך שכתוב של המערכת.

מילון למושגי השיעור

API

הכתובת הרשמית, כמו ערוץ "דיווח חדש".

JSON

שדות קבועים, שדה: ערך. הדיווח שלנו בנוי כך.

מפתח - API KEY

סודי! מצורף לכל בקשה. מקבל את היחס של זהות המדווח: שכבה נפרדת, לעולם לא בקוד.

מסלול טכנולוגי

מה קורה, מי מבצע, אם מצליח ואם נכשל. תרשים זרימה: הנוהל מצויר.

טוקן ונתב

טוקן: חלק ממילה, יחידת החיוב. נתב: חינמי או זול לשגרה, יקר לניתוח מורכב.

תיעוד

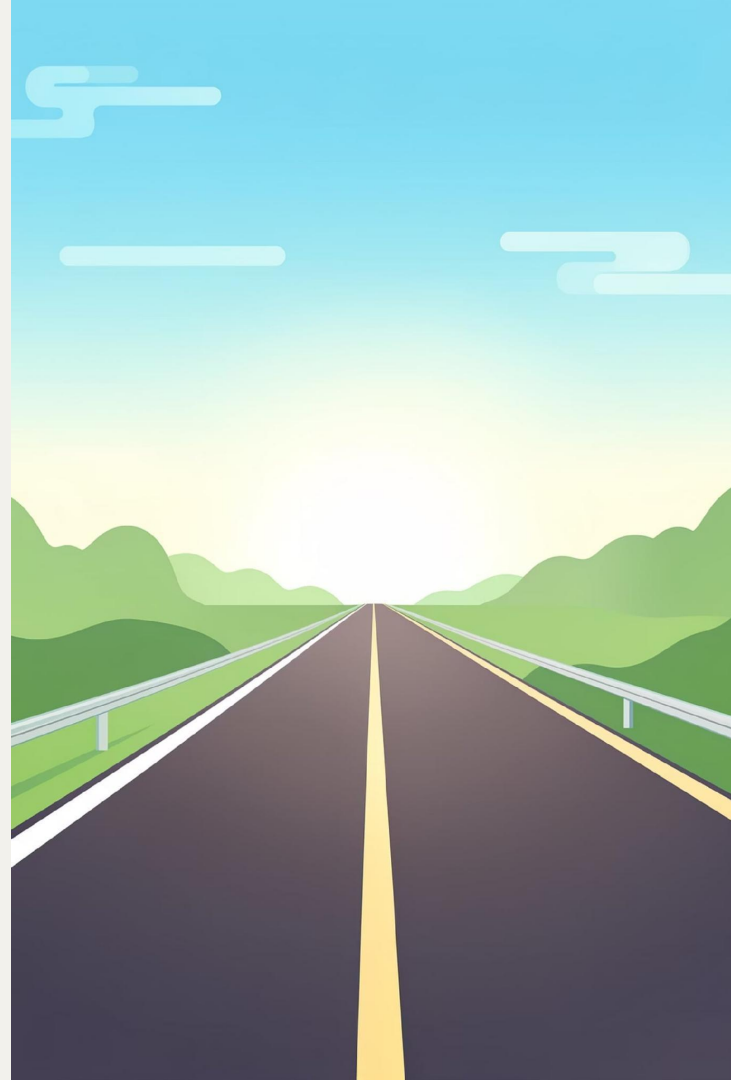
הדף שבו הספק מפרסם את שדות הטופס שלו. הוא קובע, אנחנו ממלאים אחר הדרישות.

פרומפט עבודה

נוהל כתוב ל-AI. ההפך ממלל חופשי.

מה השגנו היום

- המערכת קיבלה עוזר AI שעונה מהנהלים.
- מילאתם את הטופס הראשון שנשלח למודל.
- ירדתם שטח דיגיטלית אחרי דיווח אחד
- יצרתם לטבלת בנייה עם החלטה- עלות של כל שימוש ב AI.
- כתבתם בעצמכם את פרומפט העבודה הראשון שלכם: נוהל כתוב ל-AI.



מה יש לנו בסוף היום

ברור ERD

הישויות והקשרים, ואת ההכרעה על הטוקן האקראי.

תמונה מלאה של המערכת

שלוש שכבות, מודולים ב-FE, שירותים ב-BE, ו-Repository מעל הנתונים.

רשימה מסומנת

מה שטרם נקבע, שממתין להכרעה ולא לניחוש.

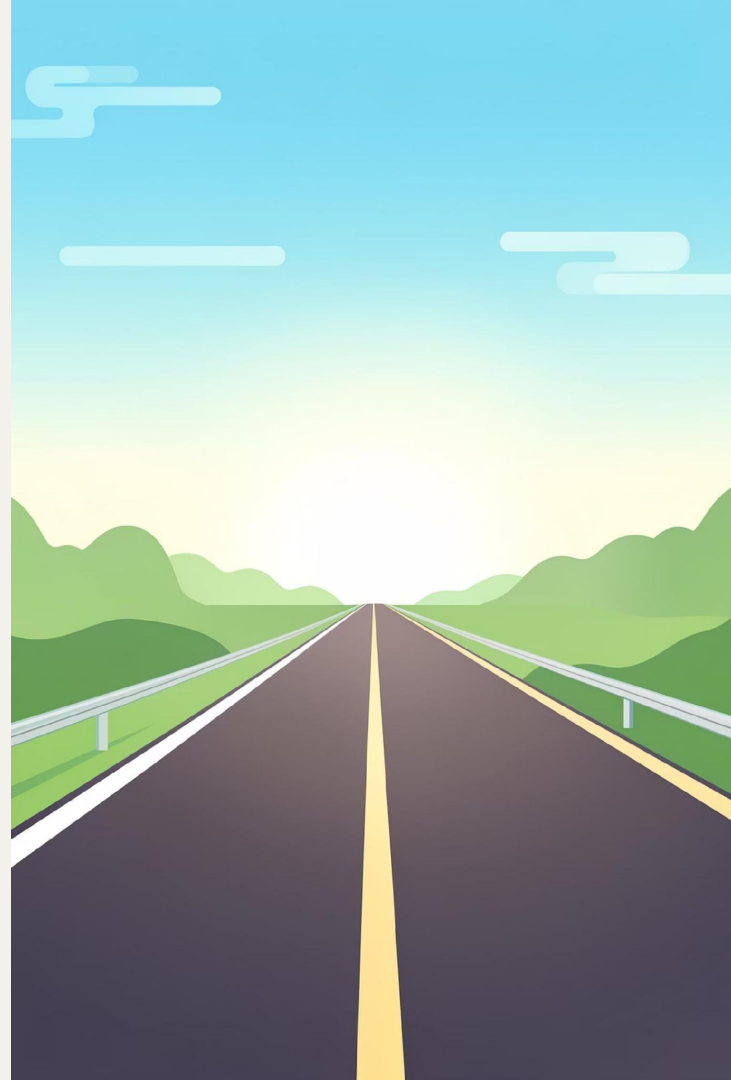
הבחנה חדשה

בין המצב היום למצב היעד, ובין מה שנחשף למה שמוגן.

מה בשיעור הבא

להריץ שוב ושוב את עדכון ה-PRD זה משעמם.
מה שמשעמם, מפסיקים לעשות ידנית.

בשיעור הבא ניצור מערכת שלומדת לעדכן את עצמה:
נבנה את שכבת הנהלים של הפרויקט
ניצורלופ שעושה את העבודה ומשאיר לכם רק את
ההכרעה.
בשלב זה הכול יתבצע בתוך הצ'אט, על הפרויקט שלכם.
בנוסף- נתכנן את הארכיטקטורה של המערכת שלנו



רטרוספקטיבה

שלוש שאלות לסיום

3

מה אקח איתי כבר מחר?

פעולה קונקרטית אחת שאוכל
ליישם מחר בבוקר בעבודה.

2

מה הפתיע אותי?

משהו שלא ציפיתי לו - תפיסה
שהתהפכה, הנחה שנשברה.

1

מה למדתי היום שלא ידעתי?

תובנה חדשה שמשנה את
האופן שבו אני חושב על ניהול
מוצר.



תודה רבה- זכרו

“

עיקרון ראשון
"ככל שקל יותר לבנות, ההחלטה האם בונים
הופכת קריטית יותר."

”

“

עיקרון שני
"בודקים את כל העלויות ולא רק את עלויות הפיתוח"

”

תודה רבה

רילי לבבי

053-4365200

levavir@gmail.com

